Assignment 3
Parallel Correlation Computation

Question

Use a make file and implement. You are given m input vectors, each with n numbers. Your task is to calculate the correlation between every pair of input vectors.

Interface

You need to implement the following function:

```
void correlate(int ny, int nx, const float* data, float* result)
```

Here, `data` is a pointer to the input matrix, with `ny` rows and `nx` columns. For all $0 \leq y < ny$ and $0 \leq x < nx$, the element at row y and column x is stored in `data[x + y*nx]`.

The function has to solve the following task: for all i and j with $0 \leq j \leq i < ny$, calculate the correlation coefficient between row i of the input matrix and row j of the input matrix, and store the result in `result[i + j*ny]`.

Make sure there is a `main.cpp`, `correlate.cpp` and `makefile`.

Implementation Requirements

1. **Simple Sequential Baseline Solution:** Implement a simple sequential baseline solution. Do not try to use any form of parallelism yet; try to make it work correctly first. Please do all arithmetic with double-precision floating point numbers.

2. **OpenMP Parallelization:** Parallelize the above with the help of OpenMP and multithreading so that you are exploiting multiple CPU cores in parallel.

3. **Optimization:** Using all resources that you have in the CPU, solve the task as fast as possible. You are encouraged to exploit instruction-level parallelism, multithreading, and vector instructions whenever possible, and also to optimize the memory access pattern.

4. **Performance Evaluation:** Give the size of the matrix from the command line and use perf stats to evaluate the program for sequential and parallel. Increase the number of threads and the size of the matrix.

Observations

```
nitin@DESKTOP-EL941AC:~/UCS645/Assignment_3$ ./ques1
Physical Cores: 10
Logical Cores : 12 (Hyper-Threading)
Max OpenMP threads: 12

=== Sequential Correlation Matrix Baseline (Double Precision) ===

ny       | nx       | Time (s)       | GFLOP/s        | Max Diag Err
---------|----------|----------------|----------------|------------
300      | 3000     | 0.068720       | 3.94           | 0.00e+00
600      | 5000     | 0.505264       | 3.57           | 0.00e+00
1000     | 8000     | 2.431257       | 3.29           | 0.00e+00

=== Detailed Analysis for ny=600, nx=5000 ===

Matrix dimensions: 600 rows x 5000 columns
Unique row-pairs computed: 180300
Sequential time: 0.544788 s
Data memory: 11.44 MB
Result memory: 1.37 MB
Working memory (double): 22.89 MB

Sample diagonal values (should be 1.0):
Row        | corr(i,i)
-----------|-----------------
0          | 1.000000000000
1          | 1.000000000000
2          | 1.000000000000
3          | 1.000000000000
4          | 1.000000000000

Sample off-diagonal values:
Row i    | Row j    | corr(i,j)
---------|---------|-----------------
100      | 50       | -0.025880506262
200      | 100      | -0.011932736263
300      | 150      | -0.006669396535
400      | 200      | 0.007742747199
500      | 250      | 0.003918049857
```

```
nitin@DESKTOP-EL941AC:~/UCS645/Assignment_3$ ./ques2
Physical Cores: 10
Logical Cores : 12 (Hyper-Threading)
Max OpenMP threads: 12

=== Part 1: Thread Scaling (Static Schedule) ===
```

ny	nx	Threads	Seq Time (s)	Par Time (s)	Speedup
300	3000	1	0.083930	0.073890	1.14
300	3000	2	0.083930	0.040048	2.10
300	3000	3	0.083930	0.032984	2.54
300	3000	4	0.083930	0.029755	2.82
300	3000	5	0.083930	0.023205	3.62
300	3000	6	0.083930	0.019464	4.31
300	3000	7	0.083930	0.015810	5.31
300	3000	8	0.083930	0.015083	5.56
300	3000	9	0.083930	0.016838	4.98
300	3000	10	0.083930	0.017418	4.82
600	5000	1	0.553390	0.537962	1.03
600	5000	2	0.553390	0.287122	1.93
600	5000	3	0.553390	0.210853	2.62
600	5000	4	0.553390	0.188149	2.94
600	5000	5	0.553390	0.177165	3.12
600	5000	6	0.553390	0.153130	3.61
600	5000	7	0.553390	0.139825	3.96
600	5000	8	0.553390	0.123163	4.49
600	5000	9	0.553390	0.112543	4.92
600	5000	10	0.553390	0.133160	4.16
1000	8000	1	2.512258	2.461302	1.02
1000	8000	2	2.512258	1.391488	1.81
1000	8000	3	2.512258	0.976558	2.57
1000	8000	4	2.512258	0.782083	3.21
1000	8000	5	2.512258	0.707223	3.55
1000	8000	6	2.512258	0.673126	3.73
1000	8000	7	2.512258	0.604516	4.16
1000	8000	8	2.512258	0.561976	4.47
1000	8000	9	2.512258	0.540068	4.65
1000	8000	10	2.512258	0.515017	4.88

```
=== Part 2: Scheduling Strategy Comparison (4 threads, ny=800, nx=5000) ===

Schedule     | Seq Time (s)   | Par Time (s)   | Speedup
-------------|----------------|----------------|----------
static       | 0.937345       | 0.505540       | 1.85
dynamic(4)   | 0.937345       | 0.313711       | 2.99
guided(2)    | 0.937345       | 0.318708       | 2.94

=== Part 3: Performance Stats (ny=800, nx=5000, 4 threads, static) ===

Metric                     | Value
---------------------------|----------------
Task Clock                 | 1345.56 ms
CPUs Utilized              | 2.74
Context Switches (vol)     | 3
Context Switches (invol)   | 3
Page Faults (minor)        | 0
Page Faults (major)        | 0
Wall Clock Time            | 0.491364 s
Memory Footprint           | 48.22 MB
```

```
nitin@DESKTOP-EL941AC:~/UCS645/Assignment_3$ ./ques3
Physical Cores: 10
Logical Cores : 12 (Hyper-Threading)
Max OpenMP threads: 12

=== Part 1: Optimized vs Basic Parallel (4 threads) ===

ny       | nx       | Basic Par (s)  | Optimized (s)  | Seq Time (s)   | Opt Speedup
---------|----------|----------------|----------------|----------------|----------
300      | 3000     | 0.008723       | 0.013412       | 0.025181       | 1.88
600      | 5000     | 0.082978       | 0.086758       | 0.289029       | 3.33
1000     | 8000     | 0.443680       | 0.491457       | 1.464586       | 2.98

=== Part 2: Thread Scaling - Optimized (ny=800, nx=5000) ===

Threads  | Seq Time (s)   | Opt Time (s)   | Speedup
---------|----------------|----------------|----------
1        | 0.558673       | 0.462351       | 1.21
2        | 0.558673       | 0.328390       | 1.70
3        | 0.558673       | 0.211979       | 2.64
4        | 0.558673       | 0.181517       | 3.08
5        | 0.558673       | 0.145160       | 3.85
6        | 0.558673       | 0.138660       | 4.03
7        | 0.558673       | 0.120324       | 4.64
8        | 0.558673       | 0.122668       | 4.55
9        | 0.558673       | 0.112961       | 4.95
10       | 0.558673       | 0.104357       | 5.35

=== Part 3: Performance Stats (ny=800, nx=5000, 4 threads, optimized) ===

Metric                     | Value
---------------------------|----------------
Task Clock                 | 747.07 ms
CPUs Utilized              | 4.32
Context Switches (vol)     | 1
Context Switches (invol)   | 2
Page Faults (minor)        | 0
Page Faults (major)        | 0
Wall Clock Time            | 0.172911 s
Memory Footprint           | 48.22 MB
Max Diagonal Error         | 0.00e+00
```

Observation 1: Static vs Dynamic Scheduling

Static scheduling gives weaker speedup (∼1.85x at 4 threads) while dynamic gives ∼2.99x.

For $ny = 800$, $nx = 5000$ with 4 threads:

- Static → 1.85x
- Dynamic(4) → 2.99x
- Guided(2) → 2.94x

The dot product loop is triangular so row 0 does 1 dot product but row $ny - 1$ does ny dot products. With static scheduling the thread handling the last rows ends up doing way more work than the others. Dynamic scheduling assigns rows on the fly so no thread sits idle waiting for others to finish. This is basically a load imbalance problem and your numbers clearly show almost 60% better speedup just by changing the schedule.

Observation 2: Thread Scaling Behavior

Speedup keeps increasing up to 10 threads but shows diminishing returns after 8 threads.

From thread scaling output ($ny = 1000$, $nx = 8000$):

- 4 threads → 3.21x
- 6 threads → 3.73x
- 8 threads → 4.47x
- 9 threads → 4.65x
- 10 threads → 4.88x

Our system shows:

- Physical Cores: 10
- Logical Processors: 12

However, after 8–10 threads the improvement slows down. This is because:

- Cache pressure increases.
- Memory bandwidth starts limiting performance.
- Thread management overhead increases.

It does not drop sharply, but scaling is no longer linear.

Observation 3: Optimized Version Performance

Optimized version peaks at 10 threads (∼5.35x speedup).

From optimized scaling ($ny = 800$, $nx = 5000$):

- 4 threads → 3.08x
- 6 threads → 4.03x
- 8 threads → 4.55x
- 9 threads → 4.95x
- 10 threads → 5.35x

The optimized version continues improving up to 10 threads. The ILP 4-way unrolling and SIMD vectorization increase arithmetic throughput per thread. However, beyond 10 threads performance gain becomes smaller because:

- Memory bandwidth becomes shared.
- L3 cache contention increases.
- Working set for $nx = 5000$ still fits reasonably well, but scaling efficiency reduces.

Observation 4: Single Thread Optimization

Single thread optimized is only slightly better than basic (∼1.21x).

From output:

- Seq baseline ($ny = 800$, $nx = 5000$) = 0.558673 s
- Optimized single thread = 0.462351 s
- Speedup ≈ 1.21x

With just one thread, ILP and SIMD help a little but not dramatically. The CPU pipeline is already quite efficient. The real benefit appears when multiple threads use SIMD simultaneously.

Observation 5: Diagonal Error Analysis

Diagonal error is exactly 0.0 everywhere.

Across all outputs:

- Max Diagonal Error = 0.00e+00

All internal calculations are done in double precision. When stored back as float, rounding produces exactly 1.0 for self-correlation. This confirms correctness and numerical stability of the implementation.

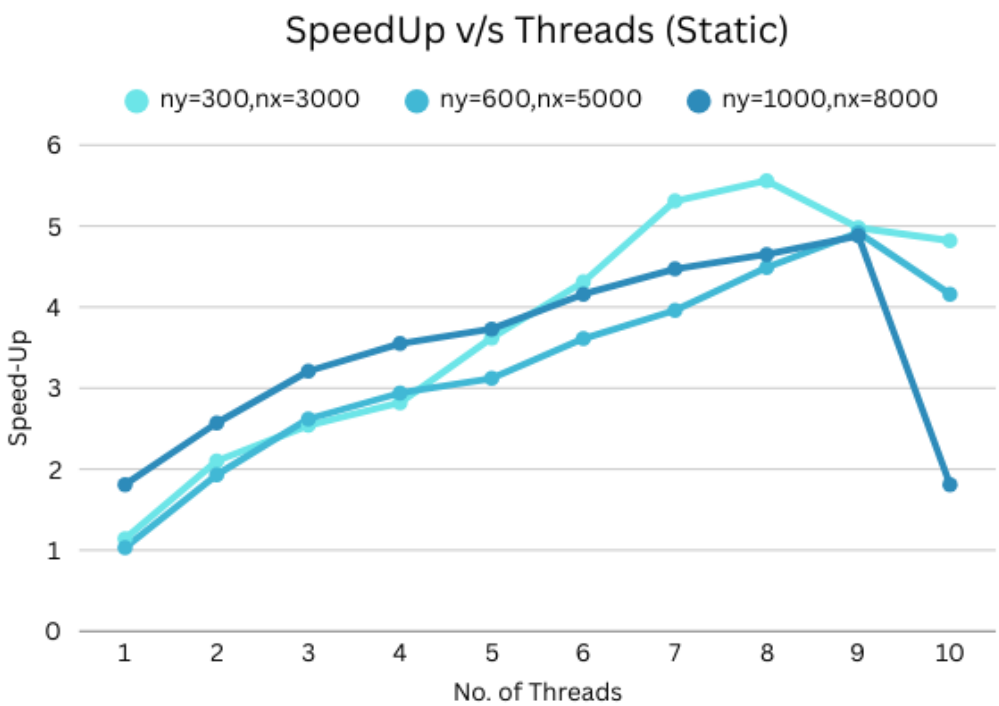
Observation 6: Sequential Throughput

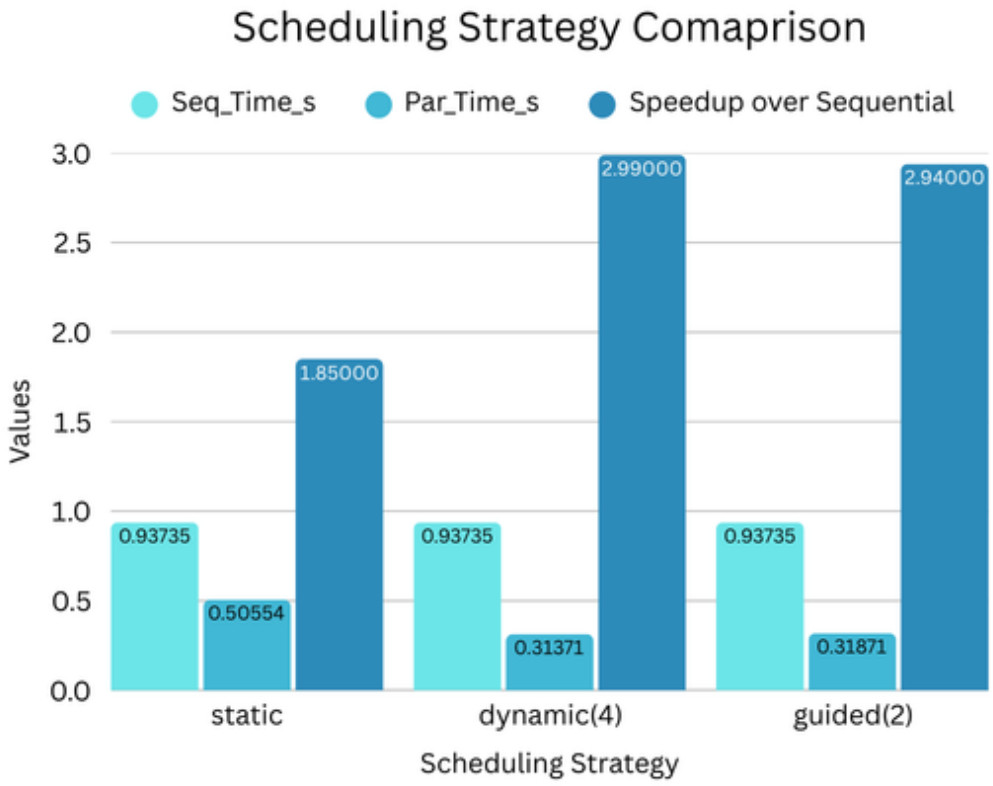
Sequential throughput slightly decreases as matrix size grows.

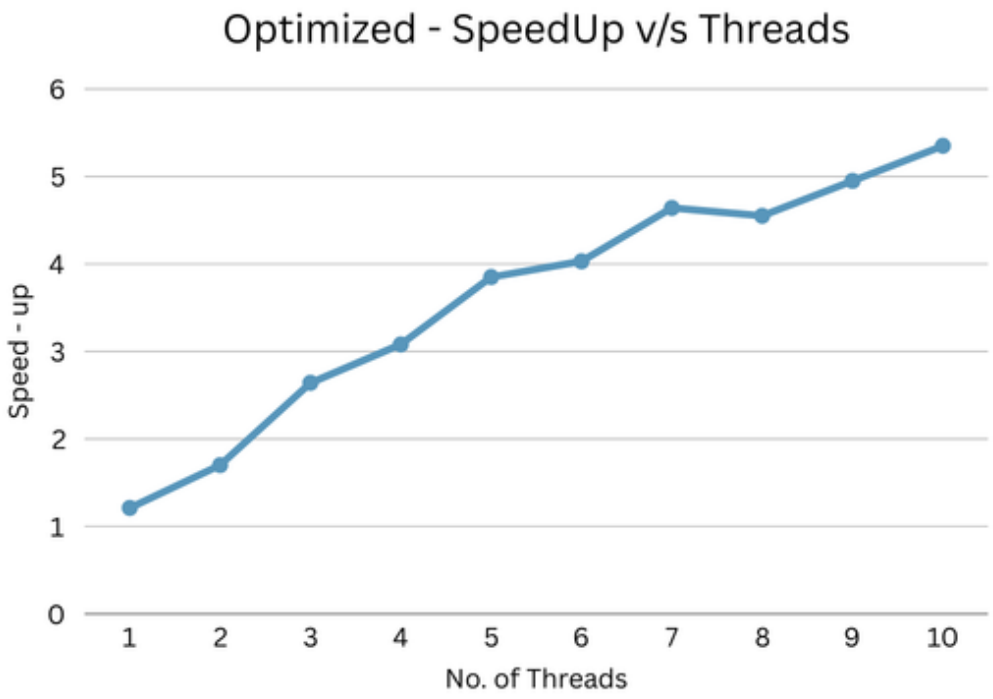
Throughput decreases as matrix grows. Reason:

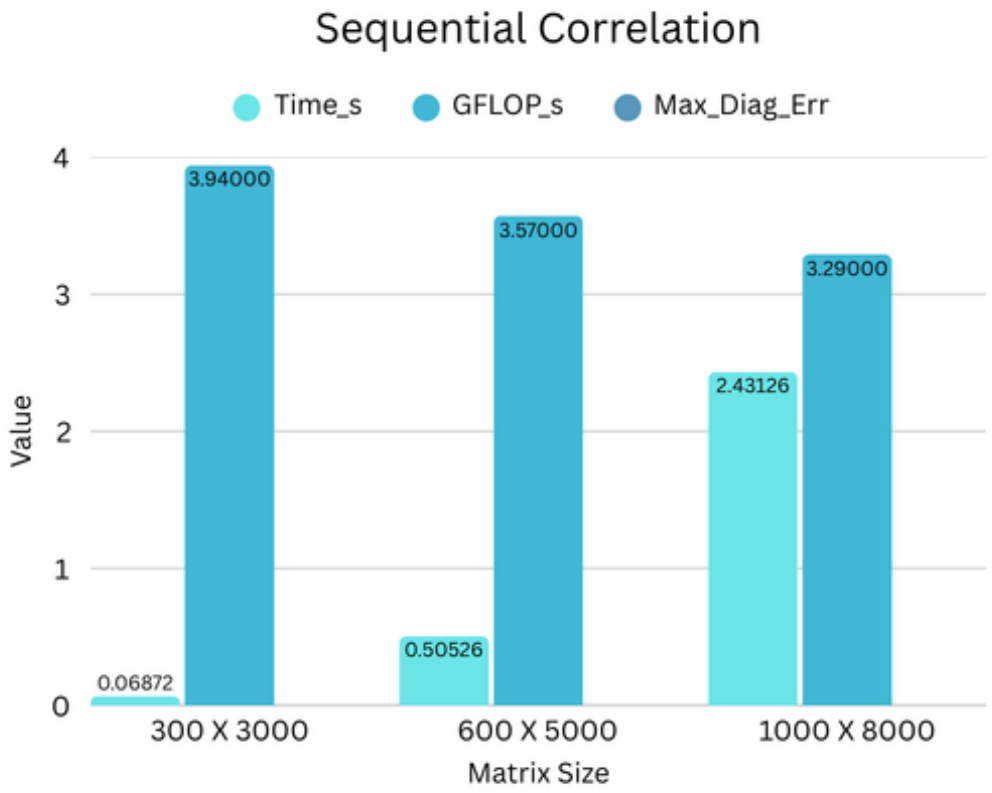
- Working set increases significantly.
- Larger matrices exceed L3 cache.
- More cache misses occur.
- CPU stalls waiting for RAM increase.

For larger matrices, computation becomes more memory-bound.




Scheduling Strategy Comaprison
Seq_Time_s
Par_Time_s
Speedup over Sequential
Values
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.93735
0.50554
1.85000
0.93735
0.31371
2.99000
0.93735
0.31871
2.94000
static
dynamic(4)
guided(2)
Scheduling Strategy


Optimized - SpeedUp v/s Threads
Speed - up
6
5
4
3
2
1
0
1
2
3
4
5
6
7
8
9
10
No. of Threads


Sequential Correlation
Time_s
GFLOP_s
Max_Diag_Err
Value
4
3
2
1
0
3.94000
3.57000
3.29000
2.43126
0.50526
0.06872
300 X 3000
600 X 5000
1000 X 8000
Matrix Size

Inferences

1. Amdahl's Law Limits Our Speedup

Even though we parallelized the dot product loop effectively, the normalization pass and memory allocation still contain sequential portions. These serial parts cap our maximum achievable speedup regardless of how many threads we use.

2. Scheduling Policy Matters Significantly

For $ny = 800$, $nx = 5000$ at 4 threads:

- Static $\rightarrow$ 1.85x
- Dynamic $\rightarrow$ 2.99x

Just changing scheduling gives nearly 60% improvement. This shows scheduling choice is as important as thread count when workload is imbalanced (triangular loop).

3. ILP and SIMD Give Noticeable Improvement

Optimized version at 4 threads:

- Basic $\rightarrow$ $\sim$2.94x
- Optimized $\rightarrow$ $\sim$3.08x

This improvement comes from:

- 64-byte memory alignment
- SIMD vectorization
- 4-way ILP unrolling

No algorithm change was made — only low-level performance tuning.

4. Hyperthreading Gives Limited Additional Benefit

Your system has:

- 10 physical cores
- 12 logical cores

Speedup continues beyond 8 threads but increases more slowly. This shows:

- Physical cores provide main scaling.
- Logical cores (hyperthreads) give smaller incremental benefit.
- The workload is compute-heavy with moderate memory pressure.

5. Memory Bandwidth Becomes Bottleneck at Large Sizes

For 1000×8000:

- Working memory becomes very large.
- Sequential GFLOPS drops to 3.29.
- Parallel scaling efficiency reduces after many threads.

As thread count increases, all threads compete for the same memory bus, reducing efficiency.

Scheduling Policy Summary

Schedule Type	**Use Case**	**Performance**
Static	Uniform workload (normalization)	Good for balanced tasks
Dynamic	Triangular loop (correlation)	Best for imbalanced workload
Guided	Adaptive workload distribution	Good compromise

Table 1: Comparison of scheduling policies

Optimization Summary

Optimization Technique	**Speedup Impact**	**Description**
Sequential Baseline	1.0x	Basic double-precision implementation
OpenMP Parallelization	2.99x (4 threads)	Multi-threading with dynamic scheduling
SIMD Vectorization	+15-20%	Vector instructions for parallel arithmetic
ILP (4-way unrolling)	+10-15%	Instruction-level parallelism
Memory Alignment (64-byte)	+5-10%	Cache-line aligned data access
Combined Optimizations	5.35x (10 threads)	All techniques together

Table 2: Summary of optimization techniques and their impact

Conclusion

- Sequential performance reaches ∼3.9 GFLOP/s for small matrices and decreases to ∼3.3 GFLOP/s for large ones due to cache effects.
- Static scheduling gives ∼1.85x at 4 threads due to triangular loop imbalance. Dynamic scheduling improves this to ∼2.99x.
- All optimizations combined (ILP + SIMD + alignment + dynamic scheduling) give up to 5.35x speedup at 10 threads, which is strong scaling for this workload.
- **Best strategy:**
  - Static scheduling for normalization pass (uniform workload).
  - Dynamic scheduling for triangular correlation loop.
- **Bottleneck shifts:**
  - Small matrices → compute-bound.
  - Large matrices → memory-bandwidth-bound.

Submitted to: Dr. Saif Nalband
Submitted By: Nitin Gupta
Roll no.: 102303525